Integrantes—

Jasiel Josue Aranda Carreras - 00000279565
Laura Fernanda Lopez Carrizosa - 00000280677
Karolina Ramos Taylor - 00000278714
Yahir Alexis Felix Balderrama - 00000281636

Asignación—
Simulación de un Sistema Operativo Básico: Gestión, Estados y Planificación de Procesos

Repositorio: https://github.com/yahirfelix22/proyecto_final

Fecha—
16 de mayo, 2026

Materia—
Sistemas operativos y arquitecturas de computadoras

Profesor—
VINKO ANTONIO NEVESCANIN MORENO

INTRODUCCION:

A veces usamos la computadora y damos por hecho que podemos tener música, el navegador con mil pestañas y un documento abierto al mismo tiempo sin que la máquina explote. Pero la realidad es que, detrás de esa 'magia', hay un jefe organizando todo: el Kernel del Sistema Operativo. El mayor reto de un ingeniero es entender cómo ese jefe decide a quién darle permiso de usar el procesador y por cuánto tiempo, especialmente cuando los recursos son limitados.

Este proyecto, titulado Soda-POP System, nace precisamente de la curiosidad de 'abrir el cofre' y ver cómo funcionan esos fierros por dentro. En lugar de solo leer la teoría, nos pusimos a programar un simulador en Python que replica las funciones más importantes de un núcleo real. Lo que hace especial a Soda-POP es que no es solo un código que corre procesos en fila, sino que implementa estructuras reales como el Bloque de Control de Procesos (PCB), que es como el acta de nacimiento de cada programa en el sistema.

A lo largo de este reporte, vamos a explicar cómo logramos que nuestra simulación pasara de lo más básico (el algoritmo FCFS, donde el primero que llega es el primero que sale) a algo mucho más avanzado y real como el Round Robin. Este último es el que de verdad nos voló la cabeza, porque usa un 'Quantum' o turno de tiempo para que ningún proceso se agandalle el procesador, permitiendo que el sistema sea justo con todas las tareas.

En resumen, este documento no es solo una entrega más; es la evidencia de cómo logramos aterrizar conceptos pesados de la carrera en un software funcional que demuestra cómo se gestiona la vida y la muerte de los procesos en una computadora moderna."

Arquitectura del Sistema:

Definición de Proceso (PCB):

Técnicamente, un proceso es la instancia de un programa en ejecución. Para gestionar estas entidades, el núcleo de nuestra simulación implementa un Bloque de Control de Procesos (PCB). El PCB actúa como la bitácora central de cada proceso, permitiendo al kernel almacenar información para suspender y reanudar tareas (Context Switch). Dentro del código Python desarrollado, esta estructura se refleja en la clase Proceso, la cual encapsula atributos clave como el Identificador (nombre), Estado actual, Tiempo total de ráfaga y Tiempo restante..

Transición de Estados:

Soda-POP implementa un modelo de estados riguroso:

LISTO (READY) →	El proceso reside en memoria esperando CPU.
EJECUTANDO (RUNNING) →	El proceso toma el control del procesador.
TERMINADO(TERMINATED) →	El proceso completa su ráfaga y sale del sistema.

Algoritmo de Planificación:

El sistema ha sido dotado con dos algoritmos distintos de planificación, lo que amplía su capacidad analítica y técnica:

FCFS (First-Come, First-Served): Un algoritmo no apropiativo que ejecuta los procesos en el estricto orden en que fueron creados. Destaca por su simplicidad.

Round Robin (RR): Un algoritmo apropiativo y avanzado gestionado a través de una cola de tipo deque. Implementa la noción de Quantum de tiempo. Si un proceso agota su quantum sin terminar su ejecución, es interrumpido y devuelto a la cola de "Listos", garantizando equidad y evitando la inanición.

Interacción y Comandos (CLI):

El sistema despliega una interfaz de consola (CLI) optimizada para la interacción mediante comandos:

Opción del Menú	Descripción Técnica
1. Crear proceso	Instancia un nuevo objeto Proceso solicitando nombre y tiempo.
2. Ver procesos	Imprime la tabla actual de procesos y sus estados.
3. Eliminar proceso	Remueve un proceso específico de la estructura de datos.
4. Ejecutar FCFS	Inicia la simulación utilizando planificación en orden de llegada.
5. Ejecutar Round Robin	Solicita un Quantum y simula la ejecución con intercambio de contexto.
6. Salir	Termina la ejecución del programa.

Evidencia de Funcionamiento:

A continuación se presenta la evidencia técnica del correcto funcionamiento del sistema y la validación de sus algoritmos:

Interfaz CLI y Ready Queue:

Se observa la inicializacion de tarea (“word” y “chrome”) con sus respectivos tiempos de rafaga y su ingreso a la taabla de control en estado “Listo”.

```
======= SISTEMA OPERATIVO SIMULACION =======
1. Crear proceso
2. Ver procesos
3. Eliminar proceso
4. Ejecutar FCFS
5. Ejecutar Round Robin
6. Salir
==========================================
Seleccione una opcion: 1
Nombre del proceso: word
Tiempo de ejecución (segundos): 5
Proceso 'word' creado satisfactoriamente.

======= SISTEMA OPERATIVO SIMULACION =======
1. Crear proceso
2. Ver procesos
3. Eliminar proceso
4. Ejecutar FCFS
5. Ejecutar Round Robin
6. Salir
==========================================
Seleccione una opcion: 1
Nombre del proceso: chrome
Tiempo de ejecución (segundos): 3
Proceso 'chrome' creado satisfactoriamente.

======= SISTEMA OPERATIVO SIMULACION =======
1. Crear proceso
2. Ver procesos
3. Eliminar proceso
4. Ejecutar FCFS
5. Ejecutar Round Robin
6. Salir
==========================================
Seleccione una opcion: 2

--- TABLA DE CONTROL DE PROCESOS ---
word       | Restante: 05s | Estado: Listo
chrome     | Restante: 03s | Estado: Listo
------------------------------------
```

Ejecución del algoritmo Round Robin:

Evidencia del intercambio de contexto (Context Switch) donde el sistema interrumpe el proceso al finalizar su Quantum y lo retorna a la cola."

```
======== SISTEMA OPERATIVO SIMULACION ========
1. Crear proceso
2. Ver procesos
3. Eliminar proceso
4. Ejecutar FCFS
5. Ejecutar Round Robin
6. Salir
==============================================
Seleccione una opcion: 5
Defina el valor del Quantum (tiempo por turno): 2

Iniciando planificacion Round Robin (Quantum: 2)...
Ejecutando word por 2s... Regresa a cola (faltan 3s)
Ejecutando chrome por 2s... Regresa a cola (faltan 1s)
Ejecutando word por 2s... Regresa a cola (faltan 1s)
Ejecutando chrome por 1s... Finalizado.
Ejecutando word por 1s... Finalizado.

Ciclo Round Robin completado.
```

Conclusión del ciclo:

Los procesos han agotado su tiempo restante (00s) y alcanzan satisfactoriamente el estado "Terminado"."

```
======== SISTEMA OPERATIVO SIMULACION ========
1. Crear proceso
2. Ver procesos
3. Eliminar proceso
4. Ejecutar FCFS
5. Ejecutar Round Robin
6. Salir
==============================================
Seleccione una opcion: 2

--- TABLA DE CONTROL DE PROCESOS ---
word        | Restante: 00s | Estado: Terminado
chrome      | Restante: 00s | Estado: Terminado
-------------------------------------
```

CONCLUSION:

"En conclusión, desarrollar este proyecto del Soda-POP System me sirvió muchísimo para entender en la práctica cómo le hace verdaderamente una computadora para no trabarse cuando tenemos varios programas abiertos al mismo tiempo. Al principio, la teoría de los estados de los procesos (Listo, Ejecutando, Terminado) sonaba un poco abstracta, pero ya al verlo correr en la terminal todo cobró sentido.

Al hacer las pruebas, me di cuenta de las diferencias reales entre los algoritmos. Por un lado, el FCFS es súper sencillo porque es literal como una fila del súper: el que llega primero, se atiende primero. Sin embargo, implementar el Round Robin fue lo que de verdad le dio el toque realista al proyecto. Ver cómo el sistema le pone 'pausa' a una tarea cuando se le acaba su tiempo (el quantum) y la manda al final de la fila para darle oportunidad a las demás, me dejó clarísimo por qué los Sistemas Operativos reales usan esta lógica para ser justos y evitar que un solo programa acapare todo el procesador.

Al final, me voy satisfecho porque el código funcionó al cien, la simulación hace exactamente lo que vimos en las clases teóricas, y creo que como equipo logramos armar un sistema operativo básico pero muy sólido."

Referencias Bibliograficas:

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Tanenbaum, A. S., y Bos, H. (2014). *Modern operating systems* (4.ª ed.). Pearson.

Python Software Foundation. (2024). *The Python language reference, version 3.12*. https://docs.python.org/3/reference/